

Definition of Done (DoD) - Music Playlist Manager

La presente Definition of Done (DoD) stabilisce i criteri formali e tecnici che ogni User Story (o task) deve soddisfare per essere considerato ufficialmente completo. L'obiettivo di questa matrice è garantire elevati standard di qualità, manutenibilità e affidabilità del software, assicurando l'allineamento con l'architettura stabilita e le metodologie Agile adottate.

1. Integrazione di Base e Versionamento

Requisiti preliminari e bloccanti per l'accettazione del codice.

- **Compilazione:** Il codice sorgente prodotto compila correttamente senza generare errori di sintassi o avvisi bloccanti all'interno dell'ambiente di sviluppo integrato (IDE) standard.
- **Git Main (Versionamento):** Il codice è stato sottoposto a commit e caricato (push) con successo sul branch designato del repository GitHub del progetto. Non vi sono file temporanei o artefatti locali pendenti.

2. Qualità della Codifica (Coding Standards)

Standard di scrittura necessari per garantire la leggibilità e la standardizzazione della codebase.

- **Nomenclatura (Naming Conventions):** Le convenzioni standard per la denominazione (es. *camelCase* per attributi e metodi, *PascalCase* per le classi) sono state applicate coerentemente in tutto il progetto. I nomi riflettono accuratamente la responsabilità logica degli elementi.
- **Formattazione:** Le regole di indentazione, spaziatura e organizzazione generale dei file sorgenti sono coerenti con le linee guida di progetto, agevolando la lettura e la revisione.
- **Assenza di Magic Numbers:** Il codice è privo di valori letterali ("hard-coded") inseriti direttamente nella logica di esecuzione. Le costanti e i valori fissi sono dichiarati esplicitamente e centralizzati (es. `public static final`).
- **Documentazione e Commenti:** Il codice è adeguatamente commentato. I commenti (incluso Javadoc per le API pubbliche) descrivono l'intento ingegneristico e le interfacce, evitando ridondanze descrittive di operazioni banali.

3. Testing Automatizzato e Affidabilità

Criteri volti ad assicurare la stabilità funzionale del dominio applicativo.

- **JUnit Automazione:** I casi di test unitari sono stati regolarmente implementati utilizzando il framework JUnit e risultano integrati nella suite di esecuzione automatizzata.
- **Copertura di Dominio:** I test validano l'interfaccia pubblica delle classi sviluppate, con un focus prioritario sui componenti del Model e sulla logica di business.
- **Validazione dei Casi Limite (Edge Cases):** Lo spazio di input è stato esplorato e testato

in modo esaustivo, includendo scenari anomali, valori limite, condizioni null e formati imprevisti.

4. Design Strutturale e Architettura (MVC)

Metriche per valutare la validità strutturale e l'aderenza al design architetturale del progetto.

- **Aderenza al Pattern MVC:** Il codice implementato rispetta rigorosamente i confini definiti dall'architettura Model-View-Controller. È garantita l'alta coesione e il basso accoppiamento (Separation of Concerns).
- **Riuso e Pulizia:** Il codice aderisce al principio DRY (Don't Repeat Yourself), prevenendo inutili duplicazioni logiche. Sono stati implementati pattern di astrazione corretti (es. State Pattern, Observer) laddove necessario.
- **Code Review Esplicita:** Il codice è stato formalmente revisionato in peer-review da un membro del team diverso dall'autore originale, al fine di individuare vulnerabilità logiche o deviazioni dagli standard.

5. Conformità al Processo Agile (Scrum)

Criteri volti ad assicurare la corretta gestione del ciclo di vita del task.

- **Aggiornamento Board (Task Trello):** Il progresso e lo stato finale del task sono stati aggiornati tempestivamente sulla Scrum Board. La scheda corrisponde allo stato reale dell'implementazione.
- **Time Tracking (Tempo Registrato):** Le ore di effort effettivo dedicate alla realizzazione del task sono state formalmente consuntivate e registrate per l'analisi della velocity di team.